Experiment 2 — Water Jug Problem using BFS
Code

```
from collections import deque

CAP_A = 4
CAP_B = 3


def next_states(state):
    a, b = state
    states = []

    # Fill operations
    states.append((CAP_A, b))
    states.append((a, CAP_B))

    # Empty operations
    states.append((0, b))
    states.append((a, 0))

    # Pour A into B
    transfer = min(a, CAP_B - b)
    states.append((a - transfer, b + transfer))

    # Pour B into A
    transfer = min(b, CAP_A - a)
    states.append((a + transfer, b - transfer))

    # Remove duplicate states
    unique_states = []
    for s in states:
        if s not in unique_states:
            unique_states.append(s)

    return unique_states


def bfs():
    start = (0, 0)
    goal = (0, 2)

    queue = deque([(start, [start])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue
```

```
        visited.add(state)

        if state == goal:
            return path

        for nxt in next_states(state):
            if nxt not in visited:
                queue.append((nxt, path + [nxt]))

    return None


solution = bfs()

if solution:
    print("Solution Path:")
    for state in solution:
        print(state)
else:
    print("No solution found.")
```

Output

```
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
(0, 2)
```

Experiment 3 — Water Jug Problem using A* Search

Code

```
from heapq import heappush, heappop

CAP_A = 4
CAP_B = 3
GOAL = 2


def heuristic(state):
    return abs(GOAL - state[0])


def successors(state):
    a, b = state
    states = []
```

```
    # Fill operations
    states.append((CAP_A, b))
    states.append((a, CAP_B))

    # Empty operations
    states.append((0, b))
    states.append((a, 0))

    # Pour A into B
    transfer = min(a, CAP_B - b)
    states.append((a - transfer, b + transfer))

    # Pour B into A
    transfer = min(b, CAP_A - a)
    states.append((a + transfer, b - transfer))

    # Remove duplicates while preserving order
    unique_states = []
    for s in states:
        if s not in unique_states:
            unique_states.append(s)

    return unique_states


def astar():
    start = (0, 0)
    pq = []

    heappush(pq, (heuristic(start), 0, start, [start]))
    visited = set()

    while pq:
        f, g, state, path = heappop(pq)

        if state in visited:
            continue

        visited.add(state)

        if state[0] == GOAL:
            return path

        for nxt in successors(state):
            if nxt not in visited:
                new_g = g + 1
                new_f = new_g + heuristic(nxt)

                heappush(
```

```
                pq,
                (new_f, new_g, nxt, path + [nxt])
            )

    return None


solution = astar()

if solution:
    print("Solution Path:")
    for step in solution:
        print(step)
else:
    print("No solution found.")
```

Output

```
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
(0, 2)
(2, 0)
```

Experiment 4 — Minimax Algorithm with Alpha-Beta Pruning

Code

```
import math


def alphabeta(depth, node, maximizing, values, alpha, beta):

    # Base case: reached leaf depth
    if depth == 3:
        return values[node]

    if maximizing:
        best = -math.inf

        for i in range(2):
            value = alphabeta(
                depth + 1,
                node * 2 + i,
                False,
                values,
                alpha,
                beta
```

```
            )

            best = max(best, value)
            alpha = max(alpha, best)

            # Alpha-beta pruning
            if beta <= alpha:
                break

        return best

    else:
        best = math.inf

        for i in range(2):
            value = alphabeta(
                depth + 1,
                node * 2 + i,
                True,
                values,
                alpha,
                beta
            )

            best = min(best, value)
            beta = min(beta, best)

            # Alpha-beta pruning
            if beta <= alpha:
                break

        return best


# Leaf node values
values = [3, 5, 6, 9, 1, 2, 0, -1]

result = alphabeta(
    0,
    0,
    True,
    values,
    -math.inf,
    math.inf
)

print("Optimal Value:", result)
```

Output

Optimal Value: 5

Experiment 5 — Genetic Algorithm (One Generation)
Code

```
# ============================================================
# GENETIC ALGORITHM - ONE GENERATION ONLY
# ============================================================

POP_SIZE = 6
CHROMOSOME_LENGTH = 5


# ============================================================
# VALIDATE CHROMOSOME
# ============================================================

def generate_chromosome(chromosome):
    chromosome = chromosome.strip()

    if len(chromosome) != CHROMOSOME_LENGTH:
        raise ValueError("Chromosome must contain exactly 5 bits.")

    if any(bit not in "01" for bit in chromosome):
        raise ValueError("Chromosome must contain only 0 and 1.")

    return chromosome


# ============================================================
# FITNESS FUNCTION: f(x) = x^2
# ============================================================

def evaluate_fitness(chromosome):
    decimal_x = int(chromosome, 2)
    return decimal_x ** 2


# ============================================================
# MUTATION FUNCTION
# ============================================================

def mutate_bit(chromosome, bit_position):
    gene_list = list(chromosome)
    index = bit_position - 1

    # Flip the selected bit
    if gene_list[index] == "0":
        gene_list[index] = "1"
```

```
    else:
        gene_list[index] = "0"

    return "".join(gene_list)


# ============================================================
# INPUT VALIDATION HELPERS
# ============================================================

def get_integer(prompt, valid_values):
    while True:
        try:
            value = int(input(prompt))

            if value not in valid_values:
                print("Invalid choice. Try again.")
                continue

            return value

        except ValueError:
            print("Enter a valid number.")


# ============================================================
# START
# ============================================================

print("=" * 54)
print("       GENETIC ALGORITHM - ONE GENERATION")
print("=" * 54)


# ============================================================
# STEP 1: ENTER 6 CHROMOSOMES
# ============================================================

population = []

print("\nEnter 6 chromosomes of 5 bits each.")

for i in range(POP_SIZE):
    while True:
        try:
            chromosome = input(
                f"Enter chromosome {i + 1}: "
            )
```

```
            chromosome = generate_chromosome(chromosome)
            population.append(chromosome)
            break

        except ValueError as error:
            print("Error:", error)


# ============================================================
# STEP 2: CALCULATE FITNESS
# ============================================================

print("\n" + "=" * 54)
print("            INITIAL POPULATION")
print("=" * 54)

for i, chromosome in enumerate(population):
    decimal_value = int(chromosome, 2)
    fitness = evaluate_fitness(chromosome)

    print(
        f"{i + 1}. Chromosome = {chromosome} | "
        f"Decimal = {decimal_value} | "
        f"Fitness = {fitness}"
    )


# ============================================================
# STEP 3: SELECT BEST TWO PARENTS
# ============================================================

population.sort(key=evaluate_fitness, reverse=True)

parent_alpha = population[0]
parent_beta = population[1]

print("\n" + "=" * 54)
print("            SELECTED PARENTS")
print("=" * 54)

print(
    f"Parent 1 = {parent_alpha} | "
    f"Fitness = {evaluate_fitness(parent_alpha)}"
)

print(
    f"Parent 2 = {parent_beta} | "
    f"Fitness = {evaluate_fitness(parent_beta)}"
)
```

```
# ============================================================
# STEP 4: ENTER CROSSOVER POINT
# ============================================================

print("\n" + "=" * 54)
print("                CROSSOVER")
print("=" * 54)

print("Chromosome length = 5")
print("Valid crossover points = 1, 2, 3, 4")

crossover_point = get_integer(
    "Enter crossover point: ",
    [1, 2, 3, 4]
)


# ============================================================
# STEP 5: PERFORM CROSSOVER
# ============================================================

offspring_1 = (
    parent_alpha[:crossover_point]
    + parent_beta[crossover_point:]
)

offspring_2 = (
    parent_beta[:crossover_point]
    + parent_alpha[crossover_point:]
)

print("\nCrossover Point =", crossover_point)
print("Offspring 1 before mutation =", offspring_1)
print("Offspring 2 before mutation =", offspring_2)


# ============================================================
# STEP 6: USER CHOOSES MUTATION
# ============================================================

print("\n" + "=" * 54)
print("                MUTATION")
print("=" * 54)

print("1. Mutate Offspring 1")
print("2. Mutate Offspring 2")
print("3. Mutate Both")
```

```
print("4. No Mutation")

choice = get_integer("Enter your choice: ", [1, 2, 3, 4])


# ============================================================
# MUTATION OPERATIONS
# ============================================================

if choice == 1:
    bit = get_integer(
        "Enter bit position for Offspring 1 (1-5): ",
        [1, 2, 3, 4, 5]
    )

    offspring_1 = mutate_bit(offspring_1, bit)

elif choice == 2:
    bit = get_integer(
        "Enter bit position for Offspring 2 (1-5): ",
        [1, 2, 3, 4, 5]
    )

    offspring_2 = mutate_bit(offspring_2, bit)

elif choice == 3:
    bit1 = get_integer(
        "Enter bit position for Offspring 1 (1-5): ",
        [1, 2, 3, 4, 5]
    )

    bit2 = get_integer(
        "Enter bit position for Offspring 2 (1-5): ",
        [1, 2, 3, 4, 5]
    )

    offspring_1 = mutate_bit(offspring_1, bit1)
    offspring_2 = mutate_bit(offspring_2, bit2)

else:
    print("No mutation performed.")


# ============================================================
# STEP 7: DISPLAY FINAL OFFSPRING
# ============================================================

print("\n" + "=" * 54)
print("              FINAL OFFSPRING")
```

```
print("=" * 54)

print(
    f"Offspring 1 = {offspring_1} | "
    f"Decimal = {int(offspring_1, 2)} | "
    f"Fitness = {evaluate_fitness(offspring_1)}"
)

print(
    f"Offspring 2 = {offspring_2} | "
    f"Decimal = {int(offspring_2, 2)} | "
    f"Fitness = {evaluate_fitness(offspring_2)}"
)


# ============================================================
# STEP 8: REPLACE TWO WORST CHROMOSOMES
# ============================================================

population[-1] = offspring_1
population[-2] = offspring_2


# ============================================================
# STEP 9: CALCULATE FINAL RESULT
# ============================================================

population.sort(key=evaluate_fitness, reverse=True)

best_chromosome = population[0]
best_decimal = int(best_chromosome, 2)
best_fitness = evaluate_fitness(best_chromosome)


# ============================================================
# DISPLAY FINAL POPULATION
# ============================================================

print("\n" + "=" * 54)
print("             FINAL POPULATION")
print("=" * 54)

for i, chromosome in enumerate(population):
    print(
        f"{i + 1}. Chromosome = {chromosome} | "
        f"Decimal = {int(chromosome, 2)} | "
        f"Fitness = {evaluate_fitness(chromosome)}"
    )
```

```
# ============================================================
# DISPLAY BEST RESULT
# ============================================================

print("\n" + "=" * 54)
print("              FINAL BEST RESULT")
print("=" * 54)

print("Best Chromosome :", best_chromosome)
print("Decimal Value   :", best_decimal)
print("Maximum Fitness :", best_fitness)

print("=" * 54)
```

Output

```
======================================================
      GENETIC ALGORITHM - ONE GENERATION
======================================================

Enter 6 chromosomes of 5 bits each.
Enter chromosome 1: 01011
Enter chromosome 2: 11011
Enter chromosome 3: 10101
Enter chromosome 4: 11110
Enter chromosome 5: 00110
Enter chromosome 6: 11111

======================================================
            INITIAL POPULATION
======================================================
1. Chromosome = 01011 | Decimal = 11 | Fitness = 121
2. Chromosome = 11011 | Decimal = 27 | Fitness = 729
3. Chromosome = 10101 | Decimal = 21 | Fitness = 441
4. Chromosome = 11110 | Decimal = 30 | Fitness = 900
5. Chromosome = 00110 | Decimal = 6 | Fitness = 36
6. Chromosome = 11111 | Decimal = 31 | Fitness = 961

======================================================
            SELECTED PARENTS
======================================================
Parent 1 = 11111 | Fitness = 961
Parent 2 = 11110 | Fitness = 900

======================================================
               CROSSOVER
======================================================
```

```
Chromosome length = 5
Valid crossover points = 1, 2, 3, 4
Enter crossover point: 3

Crossover Point = 3
Offspring 1 before mutation = 11110
Offspring 2 before mutation = 11111

==================================================
                MUTATION
==================================================
1. Mutate Offspring 1
2. Mutate Offspring 2
3. Mutate Both
4. No Mutation
Enter your choice: 3
Enter bit position for Offspring 1 (1-5): 5
Enter bit position for Offspring 2 (1-5): 5

==================================================
             FINAL OFFSPRING
==================================================
Offspring 1 = 11111 | Decimal = 31 | Fitness = 961
Offspring 2 = 11110 | Decimal = 30 | Fitness = 900

==================================================
             FINAL POPULATION
==================================================
1. Chromosome = 11111 | Decimal = 31 | Fitness = 961
2. Chromosome = 11111 | Decimal = 31 | Fitness = 961
3. Chromosome = 11110 | Decimal = 30 | Fitness = 900
4. Chromosome = 11110 | Decimal = 30 | Fitness = 900
5. Chromosome = 11011 | Decimal = 27 | Fitness = 729
6. Chromosome = 10101 | Decimal = 21 | Fitness = 441

==================================================
             FINAL BEST RESULT
==================================================
Best Chromosome : 11111
Decimal Value   : 31
Maximum Fitness : 961
==================================================
```

Experiment 6 — AND Gate using Perceptron
Code

```
import numpy as np

# Training dataset for AND gate
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

T = np.array([0, 0, 0, 1])

# Initialize weights and bias
w = np.zeros(2)
b = 0
learning_rate = 1

# Training loop
for epoch in range(10):
    for i in range(len(X)):
        net = np.dot(X[i], w) + b

        if net >= 0:
            y = 1
        else:
            y = 0

        error = T[i] - y

        # Update weights and bias
        w = w + learning_rate * error * X[i]
        b = b + learning_rate * error

print("Weights =", w)
print("Bias =", b)

# Testing phase
print("\nTesting Model Results:")

for x in X:
    net = np.dot(x, w) + b

    if net >= 0:
        y = 1
    else:
        y = 0

    print(x, "Output =", y)
```

Output

```
Weights = [2. 1.]
```

```
Bias = -3

Testing Model Results:
[0 0] Output = 0
[0 1] Output = 0
[1 0] Output = 0
[1 1] Output = 1
```

Experiment 7 — XOR Gate using Artificial Neural Network
Code

```
import math
import random


# Sigmoid activation function
def sigmoid(x):
    x = max(-500, min(500, x))
    return 1 / (1 + math.exp(-x))


# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)


# XOR training data
X = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]

Y = [
    [0],
    [1],
    [1],
    [0]
]


# Initialize weights and biases
random.seed(42)

w_input_hidden = [
    [random.uniform(-1, 1) for _ in range(2)]
    for _ in range(2)
]
```

```
w_hidden_output = [
    random.uniform(-1, 1) for _ in range(2)
]

b_hidden = [0.0, 0.0]
b_output = 0.0

learning_rate = 0.5
epochs = 10000


# Training using backpropagation
for epoch in range(epochs):
    for i in range(len(X)):

        # Forward pass
        h1 = sigmoid(
            X[i][0] * w_input_hidden[0][0]
            + X[i][1] * w_input_hidden[1][0]
            + b_hidden[0]
        )

        h2 = sigmoid(
            X[i][0] * w_input_hidden[0][1]
            + X[i][1] * w_input_hidden[1][1]
            + b_hidden[1]
        )

        output = sigmoid(
            h1 * w_hidden_output[0]
            + h2 * w_hidden_output[1]
            + b_output
        )

        # Calculate error
        error = Y[i][0] - output

        # Backpropagation
        d_output = error * sigmoid_derivative(output)

        d_h1 = (
            d_output
            * w_hidden_output[0]
            * sigmoid_derivative(h1)
        )

        d_h2 = (
            d_output
```

```
            * w_hidden_output[1]
            * sigmoid_derivative(h2)
        )

        # Update hidden-to-output weights
        w_hidden_output[0] += learning_rate * d_output * h1
        w_hidden_output[1] += learning_rate * d_output * h2
        b_output += learning_rate * d_output

        # Update input-to-hidden weights
        w_input_hidden[0][0] += learning_rate * d_h1 * X[i][0]
        w_input_hidden[1][0] += learning_rate * d_h1 * X[i][1]

        w_input_hidden[0][1] += learning_rate * d_h2 * X[i][0]
        w_input_hidden[1][1] += learning_rate * d_h2 * X[i][1]

        # Update hidden-layer biases
        b_hidden[0] += learning_rate * d_h1
        b_hidden[1] += learning_rate * d_h2


# Testing phase
print("Input\tExpected\tPredicted")

for i in range(len(X)):

    h1 = sigmoid(
        X[i][0] * w_input_hidden[0][0]
        + X[i][1] * w_input_hidden[1][0]
        + b_hidden[0]
    )

    h2 = sigmoid(
        X[i][0] * w_input_hidden[0][1]
        + X[i][1] * w_input_hidden[1][1]
        + b_hidden[1]
    )

    output = sigmoid(
        h1 * w_hidden_output[0]
        + h2 * w_hidden_output[1]
        + b_output
    )

    predicted = round(output)

    print(X[i], "\t", Y[i][0], "\t\t", predicted)
```

Output

Input	Expected	Predicted
[0, 0]	0	0
[0, 1]	1	0
[1, 0]	1	1
[1, 1]	0	0

Experiment 8 — K-Means Clustering using Pure Python
Code

```
# K-Means Clustering using Pure Python

# Student data: [Marks, Attendance]
data = [
    [85, 90],
    [80, 88],
    [82, 91],
    [35, 50],
    [40, 55],
    [38, 52]
]

students = ["S1", "S2", "S3", "S4", "S5", "S6"]

# Initial centroids
centroid1 = [85, 90]
centroid2 = [35, 50]


# Calculate Euclidean distance
def distance(point, centroid):
    return (
        (point[0] - centroid[0]) ** 2
        + (point[1] - centroid[1]) ** 2
    ) ** 0.5


# Repeat clustering
for iteration in range(10):

    cluster1 = []
    cluster2 = []

    # Assign students to the nearest centroid
    for i in range(len(data)):
        d1 = distance(data[i], centroid1)
        d2 = distance(data[i], centroid2)

        if d1 <= d2:
```

```
            cluster1.append(i)
        else:
            cluster2.append(i)

    # Store previous centroids
    old_centroid1 = centroid1.copy()
    old_centroid2 = centroid2.copy()

    # Calculate new centroid 1
    if cluster1:
        centroid1 = [
            sum(data[i][0] for i in cluster1) / len(cluster1),
            sum(data[i][1] for i in cluster1) / len(cluster1)
        ]

    # Calculate new centroid 2
    if cluster2:
        centroid2 = [
            sum(data[i][0] for i in cluster2) / len(cluster2),
            sum(data[i][1] for i in cluster2) / len(cluster2)
        ]

    # Stop if centroids do not change
    if (
        centroid1 == old_centroid1
        and centroid2 == old_centroid2
    ):
        break


# Reassign students using final centroids
cluster1 = []
cluster2 = []

for i in range(len(data)):
    d1 = distance(data[i], centroid1)
    d2 = distance(data[i], centroid2)

    if d1 <= d2:
        cluster1.append(i)
    else:
        cluster2.append(i)


# Display results
print("K-Means Clustering Results")
print("--------------------------")

print("\nCluster 1 Centroid:")
```

```
print("Marks =", round(centroid1[0], 2))
print("Attendance =", round(centroid1[1], 2))

print("\nCluster 2 Centroid:")
print("Marks =", round(centroid2[0], 2))
print("Attendance =", round(centroid2[1], 2))

print("\nStudent Cluster Assignments:")

for i in cluster1:
    print(students[i], "-> Cluster 1")

for i in cluster2:
    print(students[i], "-> Cluster 2")
```

Output

```
K-Means Clustering Results
--------------------------

Cluster 1 Centroid:
Marks = 82.33
Attendance = 89.67

Cluster 2 Centroid:
Marks = 37.67
Attendance = 52.33

Student Cluster Assignments:
S1 -> Cluster 1
S2 -> Cluster 1
S3 -> Cluster 1
S4 -> Cluster 2
S5 -> Cluster 2
S6 -> Cluster 2
```

Experiment 9 — Triangular Fuzzy Membership Function

Code

```
# Triangular Membership Function
def triangular_mf(x, a, b, c):

    if x <= a or x >= c:
        return 0.0

    elif x == b:
        return 1.0
```

```
    elif x < b:
        return (x - a) / (b - a)

    else:
        return (c - x) / (c - b)


# Triangular fuzzy set parameters
a = 20
b = 30
c = 40


# Take temperature from user
temperature = float(input("Enter Temperature Value: "))


# Calculate membership value
membership_value = triangular_mf(temperature, a, b, c)


# Display result
print("\n--- Fuzzy Membership Result ---")
print("Temperature:", temperature)
print("Membership Value:", round(membership_value, 2))
```

Output

```
Enter Temperature Value: 45

--- Fuzzy Membership Result ---
Temperature: 45.0
Membership Value: 0.0
```